SMART SOLVE CALCULATIONS

Arithmetic Parser for Larger Compiler

Software Requirements Specifications

Version 1.0

Jaiden Green

Nathan Le

Emily Messenger

Jeremiah Penney

Honey Patel

Revision History

Date	Version	Description	Author
26/02/26	1	Completed introduction (1) and scope (1.1)	Honey Patel
28/02/26	1	Completed Scope (1.2)	Honey Patel
8/03/26	1	Completed Definitions, Acronyms, and Abbreviates (1.3)	Honey Patel
11/03/26		Completed User Interface (2.1.2)	Honey Patel
12/03/26	1	Use-Case Specifications 3.2	Jeremiah Penney
12/03/26	1	References (1.4), Overview (1.4), Overall Description (2), Product perspective (2.1), assumptions and dependencies (2.5)	Nathan le
14/03/26	1	Product Functions (2.2), User Characteristics (2.3)	Emily Messenger
15/03/26	1	Completed functionality (3, 3.1, 3.1.1)	Jaiden Green
12/03/26	1	Added to functionality (3.1.2-3.1.8)	Honey Patel

© Smart Solve Calculations, 2026

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4

2. Overall Description 5
   - 2.1 Product perspective 5
     - 2.1.1 System Interfaces 5
     - 2.1.2 User Interfaces 5
     - 2.1.3 Hardware Interfaces 5
     - 2.1.4 Software Interfaces 5
     - 2.1.5 Communication Interfaces 5
     - 2.1.6 Memory Constraints 5
     - 2.1.7 Operations 5
   - 2.2 Product functions 5
   - 2.3 User characteristics 5
   - 2.4 Constraints 5
   - 2.5 Assumptions and dependencies 5
   - 2.6 Requirements subsets 5

3. Specific Requirements 5
   - 3.1 Functionality 5
     - 3.1.1 Expression Parsing
     - 3.1.2 Operator support and precedence
     - 3.1.3 Unary Operators
     - 3.1.4 Parenthesis Handling
     - 3.1.5 Numeric Constants
     - 3.1.6 User Interface
     - 3.1.7 Error Handling
     - 3.1.8 Expression Evaluation 6
   - 3.2 Use-Case Specifications 6
   - 3.3 Supplementary Requirements 6

4. Classification of Functional Requirements 6

5. Appendices 6

Software Requirements Specifications

1. Introduction

The Software Requirements Specification outlines the functional requirements and nonfunctional requirements of the arithmetic expression calculator. The functionality, use case specifications, and supplementary requirements are also defined. The purpose and scope of the SRS are outlined, along with definitions needed to interpret the SRS.

1.1 Purpose

The Software Requirements Specification outlines the functionalities required for the arithmetic expression calculator. The functional requirements, nonfunctional requirements, constraints, and other factors needed for a comprehensive understanding of the program. Further, use case specifications and supplementary requirements to provide details on system interactions and expected behavior. The product perspective explains how the calculator aligns with users, software, and external components.

1.2 Scope

The SRS applies to an arithmetic expression parser. Key functionalities include expression parsing, operator support, unary operators support, parenthesis handling, and recognizing/calculating numeric constants. The document outlines these functional requirements, along with nonfunctional requirements and constraints. Use case modeling also helps outline user interactivity with the software, and ensures each interaction is specified.

1.3 Definitions, Acronyms, and Abbreviations

Arithmetic Expression – An expression that's a combination of operands and operators (+,-,/,*,%,**) that can be calculated using PEMDAS to obtain the result.

Expression Parsing– taking in a sequence of tokens and deciding whether that sequence is valid/runnable, while also focusing on precedence and parentheses.

Tokenizer – The first stage of the compiler in which the source code is translated into executable code.

Operator- symbol that represents the operation that will be performed. %,/,*,+,-, ** will be supported in this program.

Numeric constants- constants numbers in the program that are not changed

Operator precedence- The operators are evaluated using PEMDAS rules. (Parentheses, Exponentiation, Multiplication, Division, Addition, Subtraction)

Parentheses handling- The program is able to evaluate expressions within parentheses.

Unary operator- operator that works with just one value (+/-).

Nonfunctional Requirements – Requirements that define how the functional requirements will function. For example, how fast the program should run, how much traffic it should handle, how it should look, what timeframe it should be done by, security, portability, etc.

Functional Requirements – Requirements on how the system should work, things that it should be able to handle, do, and calculate. ("WHAT the system does")

 © Smart Solve Calculations, 2026

Command Line Interface- the software through which the user can input arithmetic expressions into the arithmetic expression evaluator.

1.4 References

Software Requirements Specification References:

1. Saiedian, Hossein. *EECS 348 Term Project: Arithmetic Expression Evaluator in C++*. Spring 2026. University of Kansas.

2. IEEE Std 830-1998. *IEEE Recommended Practice for Software Requirements Specifications*. Institute of

3. Electrical and Electronics Engineers (IEEE).

4. C++ Standard Documentation (ISO/IEC 14882:2020). International Organization for Standardization.

5. Course lecture materials and assignment specifications provided for EECS 348 – Spring 2026.

1.5 Overview

The Software Requirements Specification (SRS) documents and describes the functional and unfunctional requirements for the Arithmetic Expression Evaluator project that is going to be designed in C++ language.

Section 1: Specifies the overall purpose and introduction to this project and what Use-Case models are associated with, along with terms, definitions, acronyms, and abbreviations.

Section 2: provides details and descriptions of the product that we are building, including its perspective, user characteristics, constraints, and assumptions.

Section 3: specifies the detailed functional requirements for the project. Including expression parsing, operator handling (PEMDAS), parenthesis evaluation, unary operator support, and error handling.

Section 4: classifies functional requirements by priority

2. Overall Description

The Arithmetic Expression Evaluator is a command-line C++ application designed to parse and evaluate arithmetic expressions entered by the user. The system interprets expressions containing integers, arithmetic operators, unary operators, and parentheses and computes the result entered by the user according to the defined operator while also taking in account of associativity rules (PEMDAS)

2.1 Product perspective

The Arithmetic Expression Evaluator is a standalone command-line software application developed in C++ that allows users to input and evaluate arithmetic expressions. The system does run independently and doesn't run from any outside source, databases, and network services to operate. This application that the group is going to build is going to run on a command-line interface. Users would interact within the system by putting basic arithmetic operations including addition, subtraction, multiplication, division, and follow the basic precedence rules (PEMDAS). The application will also be able to handle parentheses.

2.1.1 System Interfaces

2.1.2 User Interfaces

A command line interface is used in the arithmetic expression evaluator, which allows the user to input an expression to run through the program. The interface will support +,-,/,*,%,** operators and output a message with the result. If an invalid expression is inputted by the user, a message will pop up, and the user will be able to re-enter an input through the command line interface.

2.1.3 Hardware Interfaces

2.1.4 Software Interfaces

Software interfaces are composed of rules and protocols that enable the connection of different software components. Since our project has no clear need to exchange information between such components, no traditional software interfaces are required. However, we will need a user interface so that users can interact with the calculator (via buttons, screens, etc.) and perform desired arithmetic operations.

2.1.5 Communication Interfaces

2.1.6 Memory Constraints

A simple arithmetic calculator requires only a few numbers and functions. This should present no problem for Linux, macOS, or Windows operating systems, which usually have 8-32 GB of available RAM. However, it should be noted that a more complex graphical user interface (GUI) would consume a bit more memory.

2.1.7 Operations

2.2 Product functions

The calculator will support +,-,/,*,%,** operators, corresponding to addition, subtraction, division, multiplication, modulation, and exponentiation, respectively. After the user supplies two numbers and selects one of these operators, the calculator will process the operation and return the result to the user. In effect, proving a simple, efficient way to calculate fundamental arithmetic operations.

2.3 User characteristics

General users: People who need to perform basic arithmetic calculations (students, professionals, etc.).

Skill level: Users may have basic computer literacy but do not require programming or advanced technical knowledge. Moreover, users should be able to navigate and interact with the user interface (e.g., type numbers, select operators).

Mathematical background: Users are expected to understand basic arithmetic operations such as addition, subtraction, multiplication, division, modulation, and exponentiation. It would also be helpful to understand the basic precedence rules (PEMDAS).

2.4 Constraints

The arithmetic calculator must be programmed in C++ and must take input from the command line interface. The calculator will only support numeric constants and operators including +,-,/,*,%,**. The user must include valid expressions. Invalid expressions will not be evaluated by the program. The program should run on the KU cycle servers, which operate on a Linux

2.5 Assumptions and dependencies

This project assumes that the user will enter arithmetic expressions through the command line interface that we will prompt the user. It will also assume that the expressions are meant to follow normal order of operation rules and that parentheses and unary operators are used in a valid way. The system will depend on C++ and a normal operating system environment such as Windows or macOS in order for the system to properly compile and run. The arithmetic evaluator also depends on the tokenizer and parser working correctly so the expression can be read, checked, and evaluated properly.

2.6 Requirements subsets

3.Specific Requirements

This section describes the detailed requirements of the calculator. These requirements explain what the program is expected to do and how it should behave when interacting with the user. The purpose of this section is to clearly define the features and behavior of the calculator so it can be properly designed, implemented, and tested.

3.1 Functionality

The calculator is a program written in the C. It allows the user to perform basic arithmetic calculations by entering numbers and selecting an operation. The program processes the user's input, performs the requested calculation, and displays the result on the screen. The calculator also handles basic input errors, such as invalid operators or division by zero, by displaying appropriate error messages instead of crashing. The specific functional requirements are outlined below.

3.1.1 Expression parsing

The calculator will tokenize the input expression, including operators, numeric constants, parenthesis, and unary operators

3.1.2 Operator support and precedence

The calculator will support the following operators: +,-,*,/,%,**. Operator precedence will be implemented according to PEMDAS (parenthesis, exponentiation, multiplication, division, addition, subtraction).

3.1.3 Unary Operators

The calculator will support unary operators (+ and -).

3.1.4 Parenthesis Handling

The program will identify and evaluate expressions that include parenthesis, using them to determine evaluation order.

3.1.5 Numeric Constants

The program will identify numeric constants that are in the input. The evaluator will only handle integers.

3.1.6. User Interface

The program will provide a user-friendly command-line interface, through which the user may input their expression, and the calculated result will be outputted.

3.1.7 Error Handling

The evaluator will allow for error handling to handle and report the errors. Users may input invalid expressions (ex. dividing by zero) without the program crashing.

3.1.8 Expression Evaluation

The calculator will evaluate the expression using operator precedence, parentheses, handling, and operator evaluation to calculate the final result.

3.2 Use-Case Specifications

Use Case #1: Evaluate the mathematical expression
Actor: user

Description:
The user enters a valid mathematical arithmetic expression through the command line that will appear. The system takes that mathematical arithmetic expression and applies it to the correct operator with the correct rules. It evaluates the expression and then will show the user the result

Process:

1. The user enters a valid arithmetic expression into the first command line
2. The system reads and separates the users input into its own numbers and operators
3. The system finds the correct order of operations based on the mathematical arithmetic expression the user typed
4. The system will then calculate the final answer
5. The result is given back to the user

3.3 Supplementary Requirements

- The system needs to be in C++
- The system needs to use a command line interface
- The system needs to use the correct order of operations (PEMDAS) when evaluating the user mathematical arithmetic expression
- The system needs to be able to use the “+, -, *, /, %, **” operators

- The system needs to be able to understand how parentheses are used to insure it using the correct order of operations
- The system needs to give out a valid and correct final result back to the user after evaluating the mathematical arithmetic expression
- The system needs to only accept integer inputs

2. Classification of Functional Requirements

Functionality	**Type**
Understand and process the user's mathematical arithmetic expressions	Essential
Able to detect division (/) by zero Essential	Essential
Able to detect invalid expressions by the user Essential	Essential
Able to perform subtraction (-) operations Essential	Essential
Able to perform multiplication (*) operations Essential	Essential
Able to perform division (/) operations Essential	Essential
Able to perform modulo (%) operations Essential	Essential
Able to perform exponentiation (**) operations Essential	Essential
Correctly be able to evaluate the user expression using order of operations (PEMDAS)	Essential
Able to display clear and understandable error messages to the user	Desirable
Able to handle extra or not needed parentheses in the user's initial expression	Desirable
Able to correctly process the user's expression in a timely manner with no delays or errors	Desirable

3. Appendices

Appendix A: Example Valid Expressions

1. 6 + 7
2. 6 - (9 - 2)
3. 67 * 6/ 7
4. 6 ** 7

5. 6 * (7 + 6) % 6 – 7

5.1.1 Appendix B: Example Invalid Expressions

1. 2 * (4 + 3 - 1
2. top5 + 2
3. 4 / 0
4. 5 (2 + 3)
5. 7 & 3
6. Abs 9+10

 © Smart Solve Calculations, 2026